

Tuyển tập Bài tập: Set, Map & Balanced BST (PBDS)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng hợp Bài tập Tuyển chọn

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
1	Distinct Numbers	CSES 1621	★ Cơ bản	Đếm phần tử phân biệt bằng <code>std::set</code> $O(N \log N)$
2	Contains Duplicate	LeetCode 217	★ Cơ bản	Kiểm tra phần tử trùng bằng Set / Hash Set
3	Concert Tickets	CSES 1091	★ Cơ bản	<code>std::multiset</code> + <code>upper_bound</code> + <code>erase(it)</code> $O(M \log N)$
4	Two Sum	LeetCode 1	★ Cơ bản	Hash Map bù trừ $O(N)$ thời gian
5	Ransom Note	LeetCode 383	★ Cơ bản	Đếm phân phối tần suất bằng Map / Mảng đếm
6	Room Allocation	CSES 1164	★ Cơ bản	Sweep-line + Quản lý phòng trống bằng <code>std::set</code> / Min-Heap
7	Traffic Lights	CSES 1163	★★ Trung cấp	<code>std::set</code> tìm khoảng bao + <code>multiset</code> duy trì độ dài max
8	Nested Ranges Check	CSES 2168	★★ Trung cấp	Sắp xếp 2 chiều + Tìm kiếm cận trên Set
9	Nested Ranges Count	CSES 2169	★★ Trung cấp	GNU PBDS <code>order_of_key</code> đếm đoạn lồng nhau
10	Contains Duplicate III	LeetCode 220	★★ Trung cấp	Cửa sổ trượt kích thước K + <code>set::lower_bound</code>
11	Sliding Window Median	CSES 1076	★★ Trung cấp	Kỹ thuật Cân bằng 2 Multisets / GNU PBDS

STT	Bài toán	Nền tảng	Độ khó	Kỹ thuật trọng tâm
12	Sliding Window Cost	CSES 1077	☆☆ Trung cấp	Hai Multisets duy trì tổng nửa trái & nửa phải
13	Salary Queries	CSES 1144	☆☆☆ Năng cao	Dynamic Rank Query bằng GNU PBDS <code>order_of_key</code>
14	Josephus Problem II	CSES 2163	☆☆☆ Năng cao	GNU PBDS <code>find_by_order(k)</code> loại bỏ trong $O(\log N)$
15	Enemy is weak	Codeforces 61E	☆☆☆ Năng cao	Đếm bộ ba nghịch thế $i < j < k, a_i > a_j > a_k$ bằng PBDS
16	Sliding Window Median Hard	LeetCode 480	☆☆☆ Năng cao	Xóa trễ Lazy Removal trên 2 Priority Queues / Multisets



HƯỚNG DẪN GIẢI CHI TIẾT & GỢI Ý CHIẾN THUẬT

1. Nhóm Bài tập Cơ bản (6 bài)

Bài 1: [CSES 1621 - Distinct Numbers](#)

- **Đề bài tóm tắt:** Cho N số nguyên ($N \leq 2 \cdot 10^5$), đếm số lượng giá trị phân biệt.
- **Phân tích & Thuật toán:**
- Đưa tất cả các số vào `std::set<int>` trong $O(N \log N)$, sau đó in ra `s.size()`.
- Hoặc đưa vào `std::vector<int>`, sắp xếp rồi dùng `std::unique`.

Bài 2: [LeetCode 217 - Contains Duplicate](#)

- **Đề bài tóm tắt:** Kiểm tra xem một mảng số nguyên có chứa bất kỳ giá trị nào xuất hiện ít nhất 2 lần hay không.
- **Phân tích & Thuật toán:**
- Dùng `unordered_set<int>` duyệt qua từng phần tử. Nếu phần tử đã có trong set thì trả về `true`, ngược lại chèn vào set.

Bài 3: [CSES 1091 - Concert Tickets](#)

- **Đề bài tóm tắt:** Có N vé hòa nhạc với các mức giá $h_1, \dots, h_n$. Có M khách hàng lần lượt đến, khách thứ j sẵn sàng trả tối đa t_j . Mỗi khách mua chiếc vé đắt nhất có giá $\leq t_j$. In ra giá vé mỗi khách mua được (hoặc -1 nếu không còn vé nào phù hợp). Sau khi mua, chiếc vé đó không còn nữa.
- **Phân tích & Thuật toán:**
- Lưu toàn bộ giá vé vào `std::multiset<int> tickets`.

- Với mỗi khách hàng t_j :
 - Tìm kiếm cận: `auto it = tickets.upper_bound(t_j);`
 - Nếu `it == tickets.begin()` : Không có vé nào $\leq t_j$, in ra `-1`.
 - Ngược lại: vé phù hợp đắt nhất là `prev(it)`. In ra `*prev(it)` và **xóa vé bằng iterator**: `tickets.erase(prev(it));` (chỉ xóa đúng 1 vé trong $O(\log N)$).
- Tổng độ phức tạp: $O((N + M) \log N)$.

Bài 4: [LeetCode 1 - Two Sum](#)

- **Đề bài tóm tắt:** Tìm chỉ số của hai số trong mảng sao cho tổng của chúng bằng `target`.
- **Phân tích & Thuật toán:**
- Dùng `unordered_map<int, int>` lưu cặp (giá trị, chỉ số).
- Duyệt qua từng phần tử $A[i]$: kiểm tra xem giá trị bù $target - A[i]$ đã có trong map chưa. Nếu có, trả về cặp chỉ số; ngược lại thêm $A[i]$ vào map.

Bài 5: [LeetCode 383 - Ransom Note](#)

- **Đề bài tóm tắt:** Xác định xem xâu `ransomNote` có thể được ghép từ các ký tự của xâu `magazine` hay không.
- **Phân tích & Thuật toán:**
- Dùng mảng đếm tần suất hoặc `unordered_map<char, int>` đếm số lần xuất hiện của từng ký tự trong `magazine`, sau đó trừ dần cho các ký tự trong `ransomNote`. Nếu tần suất âm thì trả về `false`.

Bài 6: [CSES 1164 - Room Allocation](#)

- **Đề bài tóm tắt:** Có N khách đến khách sạn, khách thứ i ở từ ngày A_i đến B_i . Hai khách có khoảng thời gian ở giao nhau không thể ở chung 1 phòng. Tìm số phòng tối thiểu cần dùng và gán số phòng cho từng khách.
- **Phân tích & Thuật toán:**
- Sắp xếp các khách theo ngày đến A_i .
- Dùng một Min-Heap hoặc `std::set<pair<int, int>>` lưu các phòng đang được sử dụng theo cặp (ngày trả phòng, số phòng).
- Với mỗi khách mới: kiểm tra xem phòng có ngày trả sớm nhất có $< A_i$ không. Nếu có, khách mới dùng lại phòng đó; ngược lại cấp thêm một phòng mới `++totalRooms`.

2. Nhóm Bài tập Trung cấp (6 bài)

Bài 7: [CSES 1163 - Traffic Lights](#)

- **Đề bài tóm tắt:** Đoạn đường độ dài X . Lần lượt thêm N cột đèn giao thông tại các vị trí P_i . Sau mỗi lần thêm, in ra độ dài đoạn đường dài nhất không có đèn.

- **Phân tích & Thuật toán:**
 - Dùng `std::set<int> lights = {0, x}` lưu tọa độ các cột đèn.
 - Dùng `std::multiset<int> lengths = {x}` lưu độ dài các đoạn đường hiện có.
 - Khi thêm đèn tại P :
 - Tìm đèn liền sau: `auto it = lights.upper_bound(P);`
 - Đèn liền trước: `auto prev_it = prev(it);`
 - Xóa độ dài đoạn cũ khỏi `lengths`: `lengths.erase(lengths.find(*it - *prev_it));`
 - Thêm 2 độ dài mới: `lengths.insert(P - *prev_it); lengths.insert(*it - P);`
 - Thêm đèn vào set: `lights.insert(P);`
 - Đoạn dài nhất chính là phần tử cuối cùng của multiset: `*lengths.rbegin()`.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 8: [CSES 2168 - Nested Ranges Check](#)

- **Đề bài tóm tắt:** Cho N đoạn $[x_i, y_i]$. Với mỗi đoạn, xác định xem:
 - Đoạn đó có chứa ít nhất một đoạn khác không?
 - Đoạn đó có bị chứa bởi ít nhất một đoạn khác không?
 - **Phân tích & Thuật toán:**
 - Sắp xếp các đoạn tăng dần theo x_i ; nếu x_i bằng nhau thì giảm dần theo y_i .
 - Duyệt từ trái sang phải để kiểm tra điều kiện "bị chứa": duy trì giá trị lớn nhất của y đã gặp.
 - Duyệt từ phải sang trái để kiểm tra điều kiện "chứa đoạn khác": duy trì giá trị nhỏ nhất của y đã gặp.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 9: [CSES 2169 - Nested Ranges Count](#)

- **Đề bài tóm tắt:** Tương tự bài 2168 nhưng yêu cầu **đếm chính xác số lượng đoạn** chứa nó và số lượng đoạn bị nó chứa.
 - **Phân tích & Thuật toán:**
 - Thay vì chỉ tìm `min` / `max`, ta cần đếm số lượng phần tử y thỏa mãn điều kiện thứ tự.
 - Đây chính là bài toán đếm thứ hạng động: Sử dụng **GNU PBDS** `ordered_set` / `ordered_multiset`:
 - Dùng `order_of_key` để đếm số phần tử nhỏ hơn hoặc lớn hơn y_i trong $O(\log N)$.
 - Độ phức tạp: $O(N \log N)$.
-

Bài 10: [LeetCode 220 - Contains Duplicate III](#)

- **Đề bài tóm tắt:** Tìm xem có tồn tại hai chỉ số $i \neq j$ sao cho $|i - j| \leq \text{indexDiff}$ và $|A[i] - A[j]| \leq \text{valueDiff}$.
- **Phân tích & Thuật toán:**
- Duy trì một cửa sổ trượt kích thước `indexDiff` bằng `std::set<long long>`.
- Với mỗi phần tử $x = A[i]$, ta cần tìm xem trong set có phần tử nào nằm trong đoạn $[x - \text{valueDiff}, x + \text{valueDiff}]$ không.

- Gọi `auto it = s.lower_bound(x - valueDiff);`. Nếu `it != s.end()` và `*it <= x + valueDiff` thì trả về `true`.
- Độ phức tạp: $O(N \log(\text{indexDiff}))$.

Bài 11: [CSES 1076 - Sliding Window Median](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử và kích thước cửa sổ K . Tìm trung vị (median) của mọi cửa sổ trượt độ dài K .
- **Phân tích & Thuật toán:**
- **Cách 1 (GNU PBDS):** Duy trì một PBDS Ordered Set lưu cặp (giá trị, chỉ số). Khi cửa sổ trượt, thêm phần tử mới và xóa phần tử cũ trong $O(\log K)$, sau đó lấy trung vị bằng `s.find_by_order((K - 1) / 2) ->first` trong $O(\log K)$.
- **Cách 2 (Hai Multiset Cân bằng):** Chia cửa sổ thành hai nửa: `left` chứa $\lceil K/2 \rceil$ phần tử nhỏ hơn và `right` chứa $\lfloor K/2 \rfloor$ phần tử lớn hơn. Cân bằng kích thước hai multiset sau mỗi bước.

Bài 12: [CSES 1077 - Sliding Window Cost](#)

- **Đề bài tóm tắt:** Với mỗi cửa sổ độ dài K , tìm chi phí tối thiểu để biến đổi tất cả các phần tử trong cửa sổ thành một giá trị chung (bằng trung vị).
- **Phân tích & Thuật toán:**
- Chi phí biến đổi thành trung vị M là $\sum |x_i - M| = (\text{size}_R \cdot M - \text{sum}_R) + (\text{sum}_L - \text{size}_L \cdot M)$.
- Mở rộng thuật toán Hai Multiset từ bài 1076: Duy trì thêm tổng giá trị `sum_left` và `sum_right` mỗi khi thêm/xóa phần tử để tính chi phí trong $O(1)$.

3. Nhóm Bài tập Nâng cao (4 bài)

Bài 13: [CSES 1144 - Salary Queries](#)

- **Đề bài tóm tắt:** Có N nhân viên với mức lương ban đầu. Hỗ trợ Q truy vấn thuộc 2 loại:
- `! k x`: Cập nhật lương của nhân viên k thành x .
- `? a b`: Đếm số lượng nhân viên có mức lương trong đoạn $[a, b]$.
- **Phân tích & Thuật toán:**
- Số lượng nhân viên có lương trong $[a, b]$ bằng:

$$\text{count}(x \leq b) - \text{count}(x < a)$$

- Dùng **GNU PBDS Ordered MultiSet**:
 - `count(x <= b) = ost.order_of_key({b + 1, -1})`
 - `count(x < a) = ost.order_of_key({a, -1})`
 - Cập nhật: xóa `{old_val, id}` và thêm `{new_val, id}` trong $O(\log N)$.
- Tổng độ phức tạp: $O((N + Q) \log N)$.

Bài 14: [CSES 2163 - Josephus Problem II](#)

- **Đề bài tóm tắt:** Có N đứa trẻ xếp thành vòng tròn đánh số $1 \dots N$. Mỗi lần nhảy qua K đứa trẻ còn sống và loại bỏ đứa trẻ tiếp theo. In ra thứ tự loại bỏ ($N \leq 2 \cdot 10^5, K \leq 10^9$).
- **Phân tích & Thuật toán:**
- Đưa tất cả $1 \dots N$ vào GNU PBDS `ordered_set`.
- Vị trí cần loại bỏ tiếp theo được tính bằng công thức chia dư:

$$\text{idx} = (\text{idx} + K) \pmod{\text{s.size}()}$$

- Lấy phần tử ra bằng `auto it = s.find_by_order(idx);`, in giá trị và `s.erase(it)`.
 - Mỗi bước tốn $O(\log N)$. Tổng thời gian: $O(N \log N)$ chuẩn xác tuyệt đối!
-

Bài 15: [Codeforces 61E - Enemy is weak](#)

- **Đề bài tóm tắt:** Cho mảng N phần tử đôi một khác nhau. Đếm số bộ ba chỉ số $i < j < k$ sao cho $A[i] > A[j] > A[k]$ ($N \leq 10^6$).
 - **Phân tích & Thuật toán:**
 - Cố định phần tử ở giữa j :
 - Cần đếm số lượng $i < j$ sao cho $A[i] > A[j]$ (gọi là L_j).
 - Cần đếm số lượng $k > j$ sao cho $A[k] < A[j]$ (gọi là R_j).
 - Theo nguyên lý nhân, số bộ ba nhận j làm trung tâm là $L_j \times R_j$.
 - Dùng GNU PBDS hoặc Fenwick Tree:
 - Quét từ trái sang phải: $L_j = i - \text{order_of_key}(A[j] + 1)$.
 - Quét từ phải sang trái: $R_j = \text{order_of_key}(A[j])$.
 - Tổng đáp án: $\sum_{j=1}^N (L_j \times R_j)$.
-

Bài 16: [LeetCode 480 - Sliding Window Median \(Hard\)](#)

- **Đề bài tóm tắt:** Trả về mảng chứa trung vị thực tế (dưới dạng `double`) của mọi cửa sổ kích thước K trong mảng số nguyên N phần tử ($N \leq 10^5$, giá trị có thể đạt $2^{31} - 1$).
- **Phân tích & Thuật toán:**
- Sử dụng GNU PBDS Ordered MultiSet hoặc cơ chế Dual Priority Queues với Lazy Removal.
- Lưu ý cạm bẫy tràn số nguyên 32-bit khi tính trung bình cộng hai số nguyên lớn ở vị trí giữa: cần ép kiểu sang `double` hoặc `long long` trước khi cộng:

$$\text{median} = \frac{\text{val}_1 + \text{val}_2}{2.0}$$



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.